

```

def processar_diagnostico_rolamento():
    print("--- SISTEMA DE DIAGNÓSTICO DE ROLAMENTOS ---")

    # 1. ENTRADA DE DADOS
    try:
        horas = int(input("Digite a quantidade de horas a serem
monitoradas: "))
        if horas <= 0:
            print("A quantidade de horas deve ser maior que zero.")
            return
    except ValueError:
        print("Erro: Por favor, insira um número inteiro válido para as
horas.")
        return

    temperaturas = []
    for i in range(horas):
        while True:
            try:
                temp = float(input(f"Digite a temperatura da Hora {i+1}
(°C): "))
                temperaturas.append(temp)
                break
            except ValueError:
                print("Erro: Insira um valor numérico válido.")

    # Inicialização de contadores de estados
    alerta_baixa = 0
    ok = 0
    alerta_alta = 0
    critico = 0

    dados_relatorio = []

    # 2. PROCESSAMENTO E CÁLCULOS
    for i in range(horas):
        t_atual = temperaturas[i]

        # Classificação do estado da temperatura
        if t_atual < 70:
            estado = "ALERTA (Baixa)"
            alerta_baixa += 1
        elif 70 <= t_atual <= 90:

```

```

        estado = "Ok"
        ok += 1
    elif 90 < t_atual <= 120:
        estado = "ALERTA (Alta)"
        alerta_alta += 1
    else:
        estado = "CRÍTICO"
        critico += 1

# Variação de temperatura e vida útil do lubrificante
if i == 0:
    # Na primeira hora não há histórico anterior para calcular
    variação
    variacao = 0.0
    class_var = "NÃO HOUVE VARIAÇÃO"
    vida_util_pct = 100.0
else:
    t_anterior = temperaturas[i - 1]
    variacao = abs(t_atual - t_anterior)

    # Classificação da variação
    if 0 <= variacao <= 30:
        class_var = "NÃO HOUVE VARIAÇÃO"
    elif 30 < variacao <= 45:
        class_var = "VARIAÇÃO NORMAL"
    else:
        class_var = "VARIAÇÃO ANORMAL - CUIDADO"

    # Cálculo exponencial da vida útil do lubrificante
    vida_util_pct = 100.0 * (0.5 ** (variacao / 15))

# Guarda os dados estruturados de cada hora
dados_relatorio.append({
    "hora": i + 1,
    "temp": t_atual,
    "estado": estado,
    "var": variacao,
    "class_var": class_var,
    "vida_lub": vida_util_pct
})

# Estatísticas Globais
maior_temp = max(temperaturas)

```

```

menor_temp = min(temperaturas)
media_temp = sum(temperaturas) / horas

# 3. SAÍDA (RELATÓRIO EM TABELA)
print("\n" + "="*110)
print(f"{'RELATÓRIO DE DIAGNÓSTICO DA MÁQUINA':^110}")
print("="*110)

# Cabeçalho da Tabela
print(f"{'Hora':<6} | {'Temp (°C)':<10} | {'Estado':<15} | {'Var (°C)':<9} | {'Classificação Variação':<28} | {'Vida Útil Lub.':<15}")
print("-" * 110)

# Linhas da Tabela
for linha in dados_relatorio:
    var_str = f"{linha['var']:.1f}" if linha['hora'] > 1 else "---"
    vida_str = f"{linha['vida_lub']:.1f}%" if linha['hora'] > 1
else "100.0%"

    print(f"{linha['hora']:<6} | "
          f"{linha['temp']:<10.1f} | "
          f"{linha['estado']:<15} | "
          f"{var_str:<9} | "
          f"{linha['class_var']:<28} | "
          f"{vida_str:<15}")

print("="*110)

# Painel de Estatísticas
print(f"{'RESUMO DOS INDICADORES GLOBAIS':^110}")
print("-" * 110)
print(f" Frequência de Estados de Temperatura:           | Métricas Térmicas Gerais:")

    print(f" - Qtd. Alertas de Baixa: {alerta_baixa:<19} | - Maior Temperatura: {maior_temp:.1f}°C")
    print(f" - Qtd. Limites OK:           {ok:<19} | - Menor Temperatura: {menor_temp:.1f}°C")
    print(f" - Qtd. Alertas de Alta: {alerta_alta:<19} | - Média Aritmética: {media_temp:.1f}°C")
    print(f" - Qtd. Estados Críticos: {critico:<19} |")
print("="*110)

# Executa o programa

```

```
if __name__ == "__main__":  
    processar_diagnostico_rolamento()
```